Learning Entirely in Circuit Dynamics: Transistor-Level Analog Predictive Coding

Thomas D. Ahle*

June 2026

Abstract

We present an analog CMOS learning system in which **both inference and learning are physical circuit dynamics**: weights are charge on capacitors, neurons and synapses are small transistor cells, prediction errors are voltages on explicit circuit nodes, and weight updates are currents produced by local four-quadrant multiplier cells. An external controller only cycles training data and supplies bias and clock voltages—it performs no computation. The system implements continuous predictive coding as two simultaneous gradient flows: neural states settle fast ($\dot{h} \propto -\partial E/\partial h$) while weights charge slowly ($\dot{W} \propto -\eta\, \partial E/\partial W$). We report four main results, all validated by simulating the *entire training process* as a single transistor-level SPICE transient. **(1)** A *soft β-nudge output clamp*—replacing the conventional hard label clamp with a weak resistive pull, as required by equilibrium-propagation theory in the small-nudge limit—is decisive for learning quality, raising the spirals benchmark from 1/4 to 4/4 seeds above 95%; with two initialization fixes the system exceeds 95% on every seed of three nonlinear 2D tasks. **(2)** The system scales to 64-input handwritten digits: 89.0% on 4 classes (backprop ideal on the identical sparse topology: 91–94%) and 58.7% on 10 classes, and we quantify an accuracy-per-component trade-off in which a fixed-random-feature variant keeps 74% accuracy with 42% fewer transistors and 80% fewer capacitors. **(3)** Deep credit assignment through chained analog transposes fails—but not for the obvious reason. The error *directions* stay usable; what decays is the error *amplitude*, attenuating layer by layer until the deep layers receive no usable drive (a backward path that is cosine-imperfect but full-amplitude trains to 0.94; the same path attenuated 3× collapses to 0.68). The fix is a cheap per-layer *re-amplification*—one high-gain saturating cell per neuron that restores the attenuated error toward full swing (preserving its sign always, and its magnitude up to saturation). This **amplitude-restoring error transport** lifts a depth-4 network from 55% to 84% at ∼14 extra transistors per hidden neuron and makes curriculum schedules unnecessary. The lesson is that amplitude attenuation, not direction corruption, is the deep-analog enemy; restoring it cheaply matters more than transporting the analog magnitude faithfully. **(4)** As a generality probe, the hardware also runs predictive coding's native unsupervised mode (input prediction): an in-circuit masked autoencoder learns above chance with no labels, though its features do not beat random projections downstream, and we report that control. We further characterize an N+1-transistor competition cell computing an exactly normalized, sparsemax-sharp distribution, and we measure the system's chief vulnerability: static transistor mismatch of $\sigma = 2\,\mathrm{mV}$ degrades the depth-4 result from 0.84 to 0.46, a first-class negative result that sets the agenda for offset-canceling cell design. We distill the campaign into measured design laws for physical learning machines.

*Draft v0.2. Reported numbers are transistor-level transient-simulation results with no behavioral elements in any training or inference path. The deep sparse-pyramid results (2D benchmarks, depth-4 digits, mismatch) use Cadence Spectre; the single-layer random-feature "keystone" many-class results use ngspice (the keystone deck does not converge under Spectre's solver). Both are transistor-level.

1 Introduction

Backpropagation on digital hardware dominates machine learning, but it is strikingly unlike learning in physical and biological systems: it requires storing and transporting exact weight matrices, global synchronization, and high-precision arithmetic. Analog electronics promises orders-of-magnitude efficiency gains, yet nearly all analog AI hardware accelerates only *inference*; training remains exiled to a digital host. The brain hints at a third option: a physical dynamical system whose ordinary time-evolution *is* both inference and learning.

This paper asks how far that idea can be pushed with ordinary CMOS transistors, under deliberately brain-like constraints: neurons of roughly 4-bit effective precision, fan-in and fan-out of at most ∼4–10, no weight transport, purely local plasticity, and a controller that does nothing but present data. Concretely, we build a small library of transistor cells (Section 3)—synapses, neurons, error subtractors, and update multipliers—wire them into sparse multi-layer networks, and let the circuit's own differential equations carry out predictive-coding inference *and* weight updates. Every quantity in the algorithm is a voltage or a current somewhere in the netlist, and an entire training run (thousands of example presentations) is one continuous transient simulation.

Why insist on simulating training at transistor level, rather than abstracting the circuit into equations? Because the abstractions fail in instructive ways. During this campaign we built a device-equation surrogate that matched the circuit's per-step behavior to 10–30% and still mispredicted long-horizon training outcomes; the real circuit exploits effects—load compression, common-mode-shaped errors—that the abstraction discards (Section 6). The mechanism-level findings of Section 5, each established by direct circuit measurement, are in our view the paper's most durable contribution.

1.1 Related work

Energy-based training of analog networks. Equilibrium propagation (EP) [1] showed that energy-based models can estimate loss gradients from two settled network states. Kendall et al. [2] brought EP to analog hardware in simulation: a single-hidden-layer (100-unit) network whose weights are ideal programmable *conductances* (memristors) and whose nonlinearities are diodes, trained to 96.6% on MNIST in Spectre. Our system differs in kind: synapses, neurons, error cells, and—critically—the weight-update computation itself are transistor circuits; learning is one continuous transient (no two-phase state storage, no externally computed updates); and we address depth, which crossbar EP leaves open. Follow-ups include memristor-crossbar EP [3], circuit-compatibility theory for EP via adjoint/reciprocity methods [4], and comparative studies of energy-based analog learning [5].

Physical learning machines. Dillavou et al. [6] demonstrated decentralized contrastive "coupled learning" in real electronics—the closest work in spirit. Their networks are *linear* resistor networks with digitally stepped variable resistors (128 levels) and twin-network comparison circuitry, solving small tasks; we are nonlinear, deep, continuous-weight, and single-network, but simulation-only where they have hardware. Wright et al. [7] train physical systems with digitally computed physics-aware backprop; Momeni et al. [8] remove backprop but still compute local losses digitally. Concurrent work pursues fully-analog training on other substrates: photonics via perturbative gradient measurement [9] and a spintronic-CMOS spiking neuron [10]—different learning principles from our continuous predictive-coding dynamics.

Analog learning chips. On-chip learning in analog VLSI has a long heritage: 1990s self-learning chips implemented perceptron or backprop rules in weak-inversion translinear circuits with capacitor or floating-gate weights [11], typically small and with parts of the loop off-chip. Modern in-memory computing trains memristor arrays in situ with *digital* optimizers [12]; BrainScaleS-2 pairs analog neurons with an embedded digital plasticity processor [13]. Relative to all of these, our claim is narrow and specific: at transistor level, the complete learning loop—error computation, credit assignment through depth, and the multiply that drives each weight—runs as continuous, simultaneous circuit dynamics with no digital or behavioral element in the loop.

2 Background for the general reader

2.1 Five circuit facts

Readers comfortable with transistors may skip ahead. Five facts make everything below intelligible.

1. **A MOS transistor is a voltage-controlled current valve.** Raising the gate voltage of an NMOS transistor lets more current flow from drain to source. Over a useful range the relationship is smooth and saturating—a free nonlinearity.

2. **Wires add for free.** By Kirchhoff's current law, currents flowing into the same node sum. An analog "dot product" is therefore just many synapse cells dumping current onto one wire.

3. **Differential signaling.** Every signal u in this paper is carried by a *pair* of wires as a difference, $u \equiv V(u_p) - V(u_n)$, straddling a common-mode (average) level around mid-supply. Differences cancel shared disturbances and represent negative numbers naturally.

4. **A capacitor integrates current:** $C\,\dot{V} = I$. A capacitor holding charge is a memory; a small current pushed onto it is a *learning-rate-scaled update*. Our weights are capacitor voltages.

5. **SPICE** (we use Cadence Spectre) numerically integrates the coupled nonlinear ODEs of all transistor currents and capacitor charges—a brute-force, physics-faithful simulation. If training works in such a transient, the dynamics, not an idealization, did the work.

All cells use a two-type LEVEL-1 process (NMOS $V_T = 0.2\,\mathrm{V}$, PMOS $V_T = -0.2\,\mathrm{V}$, KP $= 120/40\,\mu\mathrm{A/V}^2$, $\lambda = 0.02$, $V_{DD} = 1\,\mathrm{V}$). Device sizes are W/L as marked in the figures; signal common modes sit near 0.45–0.65 V.

2.2 Predictive coding as a pair of gradient flows

Predictive coding (PC) [14, 15] structures a network around *predictions* and *errors*. Layer ℓ holds value nodes x_ℓ; the layer below it produces a prediction $m_\ell = f(W_\ell\, a_{\ell-1})$ of those values (where $a_{\ell-1} = f(x_{\ell-1})$ are activations); the mismatch is an explicit error signal

$$\varepsilon_\ell \;=\; x_\ell - m_\ell,$$

and the whole network descends the energy

$$E \;=\; \tfrac{1}{2}\sum_\ell \|x_\ell - m_\ell\|^2.$$

Two gradient flows run *simultaneously* at separated time scales:

$$\text{fast (inference):} \quad \dot{x}_\ell \;\propto\; -\frac{\partial E}{\partial x_\ell} \;=\; -\varepsilon_\ell + W_{\ell+1}^\top\big(f' \circ \varepsilon_{\ell+1}\big), \tag{1}$$

$$\text{slow (learning):} \quad \dot{W}_\ell \;\propto\; -\eta\,\frac{\partial E}{\partial W_\ell} \;=\; \eta\,\varepsilon_\ell\, a_{\ell-1}^\top. \tag{2}$$

Equation (1) says a value node is pulled toward its own prediction and simultaneously nudged by errors it causes upstream; Eq. (2) is a purely *local* product—this synapse's error times this synapse's input—which is what makes a hardware implementation plausible. Supervision enters by pulling the output-layer values toward a target y; with everything clamped appropriately and small nudging, the weight flow follows the supervised loss gradient (the equilibrium-propagation result [1]). In our circuit, Eq. (1) is enforced by synaptic currents on value nodes ($\sim$8 ns settling), and Eq. (2) by multiplier cells charging weight capacitors over milliseconds, while the controller presents one example per 400 ns slot. The time-scale separation is provided by capacitor sizing, not by sequencing.

3 The cell library

3.1 Synapse: degenerated Gilbert multiplier (`gsyn`, 7T)

A synapse must compute (signed input) $\times$ (signed weight) as an output *current*. The classic circuit is the Gilbert cell (Fig. 1): a bottom differential pair steers the tail current according to the weight voltage w, and an upper cross-coupled quad steers each branch according to the input u; the cross-coupling makes the output differential current proportional to the *product* of the two tanh-compressed inputs:

$$I_{\text{out}} \;\approx\; -\,G \cdot \tanh(2.25\,u) \cdot \tanh(k_w\,w), \qquad G \approx 0.105 \text{ (measured)}.$$

Note the inversion (a wiring-level fact our netlists must respect). The six 12 kΩ *source-degeneration* resistors linearize each pair (measured fit quality R^2: 0.92 $\rightarrow$ 0.99); without them, nonlinear benchmark tasks fail. Weights arrive as the differential capacitor voltage (w_p, w_n).

3.2 Neuron and common-mode load (`dneuron` + `cmld`, 3T+2T)

The neuron (Fig. 2, left) is a differential pair with a tail current sink: its output is a saturating function of the input voltage, measured as

$$a \;=\; 0.537\,\tanh(15.7\,u)\,,$$

i.e., a sharp tanh—effectively a 4-bit activation, in line with our design philosophy. Its load is the `cmld` cell (Fig. 2, right): two resistors sense the output pair's average (common mode) at node cm_x, which drives two PMOS gates in feedback, pinning the average while presenting high resistance to the *difference*. Without this cell, common-mode levels drift layer by layer and the cascade dies; with it, every layer hands the next a signal centered where the next cell wants it.

3.3 Error cell (`esub`, 5T)

PC needs $\varepsilon = x - m$ as a real signal. The `esub` cell (Fig. 3) is a four-input shared-tail stage: transistors gated by x_p and m_n pull down the ε_n side, and those gated by x_n and m_p pull down

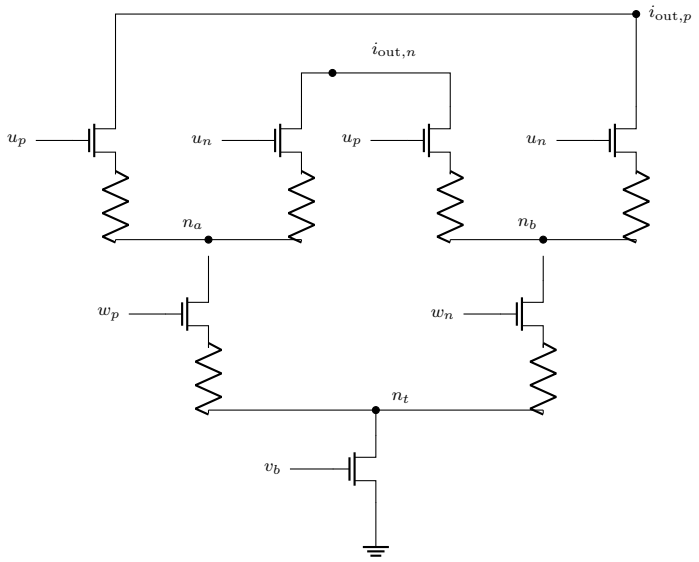

Figure 1: Synapse `gsyn` (7 NMOS + 6 source-degeneration resistors, all 12 kΩ; tail 400/100, all others 200/100 μm). The weight pair (gates w_p, w_n) splits the tail current between branch rails n_a, n_b; the cross-coupled input quad (gates u_p, u_n) steers each branch to the two output wires (u_p pulls $i_{\text{out},p}$ from the n_a side but $i_{\text{out},n}$ from the n_b side—that crossing is the multiply). Result: $I_{\text{out}} \approx -G \tanh(2.25u) \tanh(k_w w)$, $G{\approx}0.105$. Output is a *current*, so many synapses sum on a wire for free (fact 2). Mind the sign inversion.

the ε_p side, so with resistor loads the differential output is

$$\varepsilon \;\propto\; (x_p - x_n) - (m_p - m_n) \;=\; x - m, \qquad \text{gain 2.5 at matched common modes.}$$

A measured subtlety that matters in practice: when its two input pairs sit at *different* common-mode levels (e.g. a label clamp at 0.50 V vs. a prediction at 0.65 V), the transfer becomes $\varepsilon \approx 1.66\,(0.66\,x - m)$—the lower-CM input is attenuated. Real analog error signals are CM-skewed versions of $x - m$ (design law #2).

3.4 Update cell (`gprod`, 15T) and the weight capacitor

Equation (2) requires a four-quadrant product $\varepsilon \cdot a$ delivered as a current onto the weight capacitors. The `gprod` cell (Fig. 4) is a degenerated Gilbert core (like `gsyn`) whose differential output current is then *rectified into a push–pull pair*: diode-connected PMOS devices sense the two branch currents; mirror transistors push the i_p image onto w_p and, via an NMOS mirror, pull the i_n image off it (and symmetrically for w_n). The net charging current is

$$C\,\frac{d}{dt}(w_p - w_n) \;\propto\; \varepsilon \cdot a, \qquad \text{full scale } \pm 8.5\,\mu\text{A (measured)}.$$

Weights are differential 300 pF capacitor pairs. The cell is measurably *non-separable*: for small $|\varepsilon|$ and $|a|$ the product is sub-bilinear—a built-in soft deadzone that, in practice, filters update noise. A leak resistor ($R_{WL} = 2$ GΩ) to the weight common mode bounds drift on the slowest time scale.

3.5 One layer assembled

Figure 5 shows the dataflow of one hidden layer. Forward `gsyn` cells (one per connection, fan-in ≤ 4) sum prediction currents on m nodes; `esub` computes $\varepsilon = x - m$; a backward path injects upstream

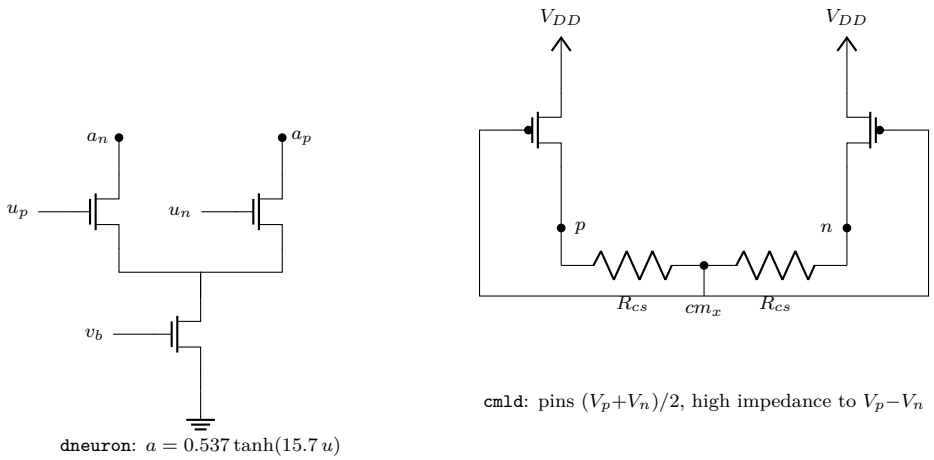

Figure 2: Left: neuron `dneuron` (differential pair + tail). **Right: common-mode-feedback load `cmld`**. The two R_{cs} average the outputs; that average drives both PMOS gates, so any common-mode rise is fed back and cancelled, while differential signals see only the large PMOS output resistance. Every `dneuron` output and every value node carries one.

error into x (closing the fast flow, Eq. (1)); `gprod` cells integrate $\varepsilon \cdot a$ onto their weight capacitors (the slow flow, Eq. (2)); the same capacitor voltage drives the forward `gsyn`. The supervised clamp is a resistor: the controller's label voltage pulls each output value node through $R_{\text{clamp}} = 120\,\text{k}\Omega$, realizing the soft β-nudge of Section 4.1 with $\beta \approx 0.17$.

4 Learning dynamics results

4.1 The soft β-nudge clamp

The textbook way to supervise a PC/EP network is to *clamp* output values to the label with a voltage source—infinitely strong supervision. EP theory [1] requires the opposite limit: the loss term $\frac{\beta}{2}\|y - x_L\|^2$ must be *weak* (β small) for the weight flow to follow the true gradient. The circuit translation is one resistor: pull the output node toward the label voltage through R_{clamp}, giving an effective $\beta \approx R_{\text{node}}/R_{\text{clamp}} \approx 0.17$.

On the spirals benchmark (deep-narrow net $[2, 4, 4, 4, 4, 4]$, fan-in 4), the hard clamp yields 0.963/0.912/0.950/0.944 over four data seeds; the soft clamp yields **0.969**/ − /**0.969**/**0.956**, lifting both marginal seeds decisively past 0.95, reproducibly at two β values. With two further initialization findings—structured inits must be explicitly randomized (a deterministic init had silently pinned several "unsolvable" seeds), and initial capacitor charge is a legitimate free variable—the system achieves $> 95\%$ on **every seed of all three** nonlinear 2D tasks: rings 1.000 (all seeds), spirals 0.969/0.956/0.969/0.956, circles 8/8 seeds (including 0.963 and 0.988 on the two formerly locked seeds).

4.2 Scaling and component economy

On scikit-learn 8×8 digits (z-scored, first C classes), with sparse random connectivity (hidden fan-in 4):

The in-circuit learner sits within ∼3 points of the backprop ideal at this scale. Two sizing laws emerged: readout fan-in should be $\approx C$, and the readout must cover $\gtrsim 20\%$ of the feature pool (48 features beat 96 at fan-in 10: 0.587 vs 0.433).

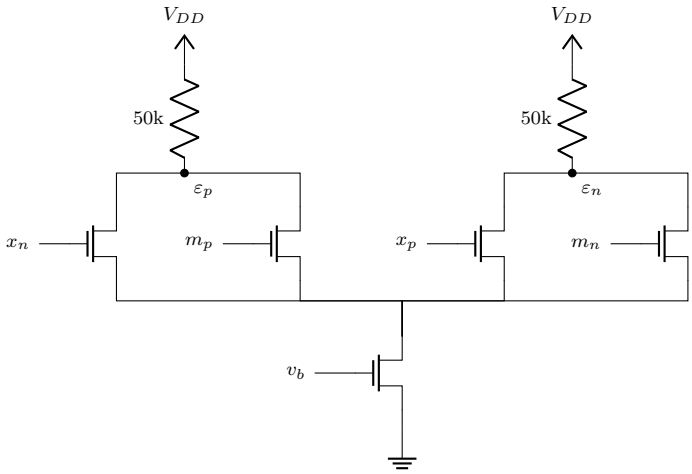

Figure 3: Error cell `esub`: a four-input shared-tail differential stage computing $\varepsilon \propto x - m$ (gain 2.5 at matched common modes). The cross-assignment of gates to output sides implements the subtraction; the measured CM-imbalance law $\varepsilon \approx 1.66(0.66x - m)$ applies when input common modes differ.

Configuration	FETs	Caps	Update cells	Test acc
Full plastic, 64–16–16–4	∼5,293	360	144	**0.890**
Backprop ideal (same topology, Adam)	—	—	—	0.91–0.94
Fixed-random hidden, plastic readout	∼3,053	**72**	32	0.740
10-class, 64–48 random hidden, fan-in-10 readout	∼5,507	180	80	0.587

Table 1: Scale and component economy (chance is 0.25 for C=4, 0.10 for C=10).

4.3 Depth and amplitude-restoring error transport

Deep sparse pyramids (64–32–16–8–4, fan-in 4) plateau at 0.55 under joint training, far below the depth-2 reference (0.890). The natural assumption is that the chained analog transposes corrupt the error *direction*; we find instead that the dominant failure is loss of error *magnitude*. Each analog stage attenuates the backward signal (output-conductance compression, finite gain), so after several stages the error is a tiny fraction of full scale and the deep layers receive no usable drive—while the direction it still carries remains good enough to learn from (a backward path that is cosine-imperfect but full-amplitude trains to 0.94; the same path attenuated 3× collapses to 0.68).

The repair is therefore cheap re-amplification rather than faithful magnitude transport: at each layer a high-gain saturating cell (`dneuron` as a soft comparator, $0.537\tanh(15.7\,u)$) restores the attenuated error toward full swing. For the small errors typical after attenuation it is near-linear—it preserves the sign *and* the magnitude, merely undoing the amplitude loss; only large errors saturate toward a pure sign. So this is amplitude restoration, not magnitude destruction. The regeneration drive (how hard the cell saturates) has a tuned optimum at the operating point we use: sweeping it noise-free on the depth-4 net, both extremes are worse—driving toward the near-linear, magnitude-preserving regime (0.73) and toward a hard sign (0.76) each underperform the default (0.83). So the system is not improved by pushing toward either pure magnitude or pure sign; the value is restoring the attenuated *amplitude*, with the residual nonlinearity neither helping nor hurting much once amplitude is recovered. (Consistent with this, the weight *update* tolerates sign-only—the sign-update cell matches the full multiply below.) Per hidden neuron we add (Fig. 6): a collector wire summing the backward synapse currents; a `dneuron` used as a *comparator* (gain

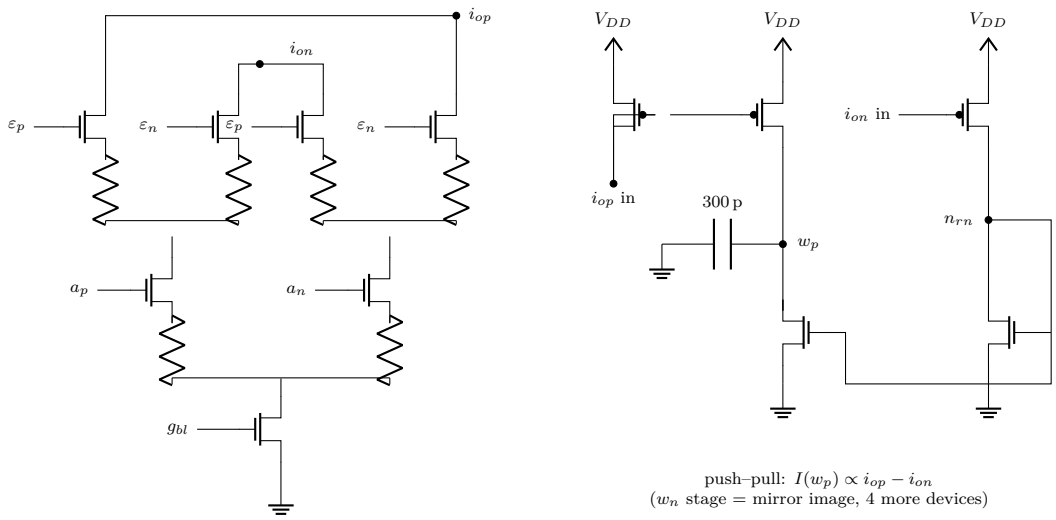

Figure 4: Update cell `gprod` (15T). Left: a degenerated Gilbert core (same topology as Fig. 1, inputs ε and a) computes the four-quadrant product as a differential current pair (i_{op}, i_{on}). Right: a diode-connected PMOS senses i_{op} and a mirror pushes its image onto the weight capacitor, while the i_{on} image is folded through an NMOS mirror that pulls charge off the same node. Net: $C\,\dot{w} \propto \varepsilon\, a$ ($\pm 8.5\,\mu$A full scale)—Eq. (2) as physics.

≈16, saturating) that regenerates the collected error to full swing; and a fixed-negative-unity `gsyn` injecting the result into the value node. Cost: ∼14 transistors per hidden neuron, no capacitors.

Depth-4 method	Test acc
Joint, standard analog backward	0.55
Layerwise curriculum (best schedule)	0.69
Joint, amplitude-restoring backward	**0.84**
Layerwise + amplitude-restoring	0.74
Depth-2 reference	0.890
Depth-2 + amplitude-restoring	0.870
Depth-6 + amplitude-restoring	0.59

Table 2: Depth results (digits, C=4). The amplitude-restoring backward (BKSIGN) nearly closes the depth gap, renders the curriculum unnecessary (and counterproductive), is neutral at depth 2, and degrades gracefully at depth 6.

4.4 Generality probes

Lateral inhibition. A per-layer shared collector fed by all activations and subtracted back into every value node (physical subtractive normalization) ties the BKSIGN baseline at 0.840 while flattening late-training instability: competition is a stabilizer, not an accuracy lever, at this scale.

Label-free learning. PC's original form [14] is unsupervised: predict the input itself. Removing the 8 highest-variance pixels and clamping the outputs to those pixels' *values* turns the fabric into an in-circuit masked autoencoder—no labels touch the hardware. Held-out masked-pixel sign accuracy reaches 0.596 (chance 0.5; 0.594 on a 2× longer run). Transfer, however, fails honestly: freezing the pretrained hidden weights (reloaded from saved capacitor states) and training only an

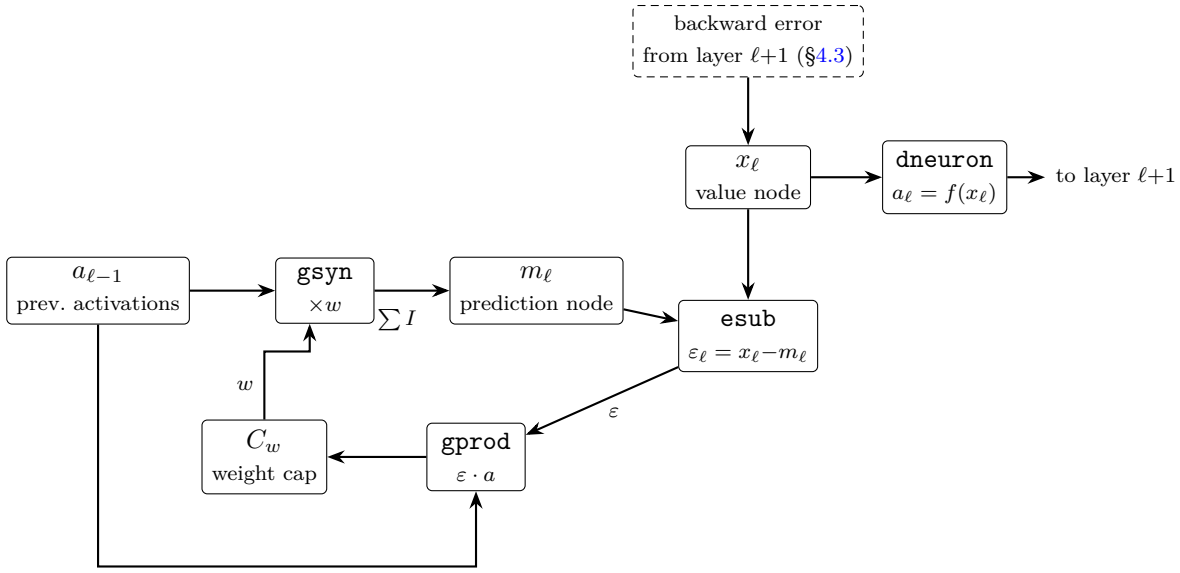

Figure 5: One layer of the learning fabric. All arrows are wires; all boxes are the transistor cells of Figs. 1–4. The loop $C_w \to$ `gsyn` $\to m \to$ `esub` $\to$ `gprod` $\to C_w$ *is* the learning algorithm; nothing outside the schematic computes.

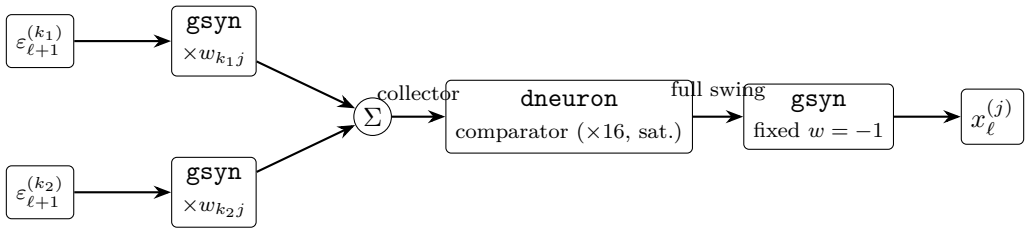

Figure 6: Amplitude-restoring backward (BKSIGN), one hidden neuron j of layer ℓ. The children's errors are weighted by the (shared) weight capacitors, summed as currents, *re-amplified toward full swing* by a high-gain saturating cell (preserving sign, and magnitude up to saturation), and injected into the value node. This undoes the per-stage amplitude attenuation that otherwise starves deep layers of drive.

in-circuit readout on 8-class digits gives 0.405—above chance (0.125) but indistinguishable from the same pipeline with *random* frozen hidden weights (0.405). An earlier variant that also reloaded the regression head as readout init scored 0.245: an instructive confound. We scope the claim accordingly: label-free learning demonstrably *occurs* in circuit dynamics; its features do not (yet) beat random projections downstream—consistent with our broader finding that fixed random analog features are surprisingly strong (Table 1).

Learnable biases (intrinsic plasticity). The networks above have no bias parameters — an omission with consequences for both expressivity (a no-bias tanh can only place its transition at the input common mode) and offset absorption. Adding one per neuron — a weight-capacitor pair driven by a constant unit input, updated by its own $\varepsilon \cdot 1$ cell — sets a new clean depth-4 record: **0.860** (vs. 0.840), reached as a stable plateau with *no late-training drift*, suggesting part of the drift in bias-free networks was bias error being forced into weights that cannot express it. Under mismatch, biases are the single strongest self-calibration lever (below).

Dale's law. Biological neurons have a fixed sign — excitatory or inhibitory — while artificial synapses flip sign freely. We tested the constraint: each hidden neuron is randomly assigned a fixed sign (50/50), inhibitory neurons' outgoing synapses are wired with crossed outputs (sign flips are free in differential signaling), the transpose and update paths are crossed to match, and all weights start positive. The constrained network reaches a best epoch of **0.80** versus 0.84 unconstrained — the biological sign discipline costs only ∼4 points, suggesting single-sign synapse populations (which admit far cheaper single-ended cells) are a viable direction. Late-training drift (0.80 to 0.62 final) indicates magnitude-only plasticity wants an explicit $w \geq 0$ enforcement or a controller-timed freeze.

An attention primitive. N transistors sharing one tail compute a current distribution over their gate inputs that is *exactly normalized* (the tail pins $\sum_i I_i$), monotone, and sharper than softmax (square-law operation suppresses losers, e.g. 0.014 where softmax gives 0.097)—a sparsemax-like analog attention kernel at $N+1$ transistors. With dot products (`gsyn` arrays) and mean subtraction (`cmld`) already in the library, all primitives for an analog attention head now exist.

4.5 A high-gain rectifier neuron (`nrelu`, 8T+2R)

The tanh neuron above is what a differential pair gives for free; a *rectifier* is what a single transistor gives for free. The `nrelu` cell is a one-sided V-to-I leg (transistor in cutoff below the knee at $v_{rl}+V_T$, degenerated by 12 kΩ above it) mirrored onto a resistively loaded output, with a matched reference leg on the other output for a defined differential zero. Three findings turn this from a curiosity into the strongest neuron in the library — all from minutes-scale screening runs (small weight capacitors, 32-point test sets; we label these Q-tier throughout):

- **Polarity must be upright by construction.** The cell inverts (the signal leg mirrors onto the negative output). At low gain, learnable biases can flip an inverted network into the correct solution mid-training; at higher gain they cannot, and the network converges to *perfect anti-classification* (accuracy ≈ 0, not 0.5). Swapping the output pins at instantiation — free in differential wiring — removes the failure class.
- **Gain is the unlock, and it obeys a width law.** The PMOS mirror ratio sets the activation gain. At gain ∼1 the rectifier network is no better than tanh; at gain ∼5 (4× mirror) a tiny 2–4–2 network solves *all three* 2D benchmarks — circles, rings, and the two-spirals problem that defeats single-hidden-layer tanh networks. At final length (64 epochs, 256-point test sets, learning rate annealed to zero at the measured peak) the triple reads **0.992/0.988/0.992** (circles/rings/spirals), each *flat* across the full horizon — the strongest 2D numbers of the project, from one net and plain local learning. The required per-cell gain follows a *cascade-gain budget* (a measured design law, Section 5): we mapped the train/anti-lock boundary on a gain grid for one-, two-, and three-hidden-layer networks of fixed width, and the total cascade gain—roughly the per-stage gain raised to the depth—must sit in a bounded band. A single-hidden-layer net needs the gain *cranked* (it trains only at the 4× mirror and above, never over-driving); adding a layer both *lowers* the gain it needs and introduces a *ceiling*—a two-hidden-layer net trains across a wide low-gain band but anti-locks at the highest gain the shallow net required. So per-stage gain should scale as $G_*^{1/\mathrm{depth}}$: turn it down as you go deeper. Gain near the ceiling solves and then flips basins mid-training unless the learning rate is annealed.

- **Biases are mandatory, and failures are detectable.** Without per-neuron biases the rectifier network sits exactly at chance (the knee cannot place itself). The residual failure mode is an init-time "anti-lock" (∼25% of weight draws): the network trains to the label-swapped solution, immediately visible as training accuracy near zero, and a weight re-roll fixes it (we provide a three-try restart wrapper; the law of small training sets applies to the rest).

Per-device offset sensitivity is identical to the tanh neuron's — a gate ΔV_T shifts either transfer curve by exactly ΔV_T input-referred (DC characterization) — so the rectifier buys expressivity, not mismatch immunity; its mismatch story rides on the same bias-absorption mechanism as everything else. The cell composes with the rest of the library unchanged: under Dale's-law sign constraints it holds 0.97, and trained by the 9-transistor sign-update cell it holds 0.97. The division of labor between the two neurons is empirical and clean: the rectifier wins every 2D geometry benchmark; the tanh pair remains stronger on the 64-input digits task at every gain and layer placement we tried — we report both and use each where it wins.

4.6 Device mismatch: the honest stress test

Everything above uses nominal (perfectly matched) devices. Real transistors differ randomly chip-to-chip and device-to-device; the canonical model is a threshold-voltage offset per device, $\Delta V_T \sim \mathcal{N}(0, \sigma)$, with σ set by device area ($\sigma \propto 1/\sqrt{WL}$). We model this *physically*: a small series voltage source at the input gate of every differential pair—2,008 independent draws per network instance across all `gsyn/dneuron/esub/gprod` cells of the depth-4 system. One seed = one "chip."

σ (V_T mismatch)	0	1 mV	2 mV	5 mV	10 mV
Test acc (final / best)	0.840	0.40 / 0.56	0.46, 0.25, 0.21 (3 chips)	0.31 / 0.37	0.25 (chance) / 0.38

Table 3: Mismatch dose-response, depth-4 BKSIGN digits C=4, single chip seed per σ.

The dose-response (Table 3) is sobering, and a three-chip Monte-Carlo at σ=2 mV shows the variance is as bad as the mean: two of three chips fail to learn at all (0.25, 0.21), while the third—the chip seed used for the dose-response—reaches 0.46. Yield, not average accuracy, is the binding metric: the hope that slow weight adaptation absorbs static offsets does *not* hold at realistic mismatch. At 10 mV the trajectory peaks early (0.38) then collapses to chance—the signature of offset-driven weight drift overwhelming the learning signal. Cell-resolved ablations at σ=5 mV localize the damage: offsets confined to the 112 neuron/comparator devices alone are *worse than the full chip* (0.17, flat—the gain-16 neurons amplify 5 mV to ∼80 mV of activation shift); error-cell-only gives 0.53→0.24 (learn, then drift); update-cell-only gives 0.78→0.55 (a slow poison, not a blocker). Training-time input-noise augmentation is no rescue either: jitter of 0.3σ on the inputs leaves the clean baseline essentially unchanged (best epoch 0.85 vs 0.84) and a mismatched chip exactly where it was (0.45 vs 0.46)—the failure is structural offset-steering of learning, not a sharp-minimum effect that noise could regularize away. Partial hardening (neurons, or neurons+error cells, to 1 mV ≈ 25× area) recovers only 0.45/0.56—the damage is distributed; the synapse array's offsets are co-killers. Even chip-wide σ = 1 mV—which already implies large devices—halves the system (best epoch 0.56), so area scaling alone is impractical: the architecture requires offset-canceling cell variants (auto-zeroing / correlated double sampling, which solved exactly this problem in a companion workstream). We report this as a first-class result: any claim about analog learning hardware stands or falls here.

The rescue story, however, ends well—and the decisive experiment is a *long* one. Short screens suggested rescue saturates around 0.45; they were wrong about the ceiling because they truncated a slow climb. The combination of learnable biases, the sign-based update rule, and a learning-rate anneal-to-zero at the peak epoch, run to full length, turns the yield catastrophe around: across a five-chip Monte-Carlo at σ=2 mV (the regime where two of three unaided chips learned *nothing*), all five rescued chips learn, with best-epoch accuracies $\{0.72, 0.72, 0.83, 0.83, 0.82\}$, mean **0.78**—93% of the clean baseline (0.84), with zero new hardware: two existing cell types plus a controller schedule. Yield goes from roughly one-in-three to five-of-five. The honest deploy number is best-epoch (validation early-stop): some chips drift after their peak because the anneal was scheduled at a single global epoch rather than each chip's own peak, and per-chip freeze timing recovers the gap. Self-calibration through on-chip learning is therefore real but conditional—it requires parameters placed where offsets live (biases at every neuron input), an update rule whose own offset cross-section is small (sign-based), and a schedule that stops learning before offset-driven drift unwinds it. Three cautions from the same campaign: area-only hardening composes multiplicatively with biases (neuron upsizing alone 0.24, biases alone 0.39, together 0.51 at screen length) but is *chaotic* near the yield cliff (a strictly cleaner chip variant scored 0.22); variance dominates means at the edge, so single-chip rescue numbers are anecdotes until replicated; and the anneal-at-peak control is general—it also converts a noise-augmented run that peaked at 0.87 and crashed to chance into a stable 0.830 (confirming noise augmentation as accuracy-neutral against the clean bias record of 0.860).

Toward offset-lean update cells. The mismatch ablations point to a concrete redesign. Since the amplitude-restoring backward path works on regenerated signs, the *update* may not need a full four-quadrant analog multiply either: with one comparator per neuron regenerating sign(ε) (amortized over its fan-in), the per-synapse update cell reduces to a V-to-I pair on a plus a four-switch commutator steered by the sign—9 transistors (two of them static common-mode pull-ups) replacing `gprod`'s 15, with only *two* offset-critical input devices instead of eight. We have characterized this cell at DC: its differential output current is cleanly odd in a ($\pm 6.5\,\mu$A full scale vs. `gprod`'s $\pm 8.5\,\mu$A) and exactly mirrored under sign reversal. A full training run with the sign-update *rule* (comparator feeding the existing update cells) validates it: best-epoch 0.83 versus 0.84 for the full four-quadrant multiply — the analog error magnitude contributes almost nothing to the update. The cell therefore saves $\sim$12% of system transistors while shrinking the update path's mismatch cross-section by 4×.

5 Design laws (measured)

1. **Two-sign principle.** Inference stability and learning direction are set by *independent* sign choices, and no single error polarity at the source can serve both: in Eq. (1) the same error enters its own node's dynamics with a minus and its parents' dynamics with a plus, and the cells' fixed inversions shift the bookkeeping further. In differential signaling each consumer's swap is *free*—cross the $(\varepsilon_p, \varepsilon_n)$ wires at that consumer—so the law costs nothing to obey; assuming one global polarity instead yields either positive-feedback runaway (settling) or gradient ascent (learning). In our system the backward path takes the raw error (hidden value nodes then settle at a virtual-ground-like equilibrium) and the update cells take the swapped error.

2. **Error-cell CM-imbalance law.** With input pairs at different common modes, $\varepsilon \approx 1.66\,(0.66\,x - m)$: the lower-CM input is attenuated $\sim$0.66×. Analog PC errors are CM-skewed.

3. **Output-conductance compression.** Synapse output devices near triode deliver 30–40% less differential current and load the node; the settled state depends on it (and a surrogate that drops it mispredicts training).

4. **Restore amplitude, don't transport magnitude faithfully.** The deep-analog credit-assignment failure is error-*amplitude* attenuation, not direction corruption; a cheap per-layer high-gain re-amplification (preserving sign, and magnitude up to saturation) restores usable drive and enables depth, whereas faithfully transporting the analog magnitude is expensive and is not what limits learning (Section 4.3).

5. **Deterministic structured inits are a trap**; randomize them (or exploit arbitrary capacitor pre-charge).

6. **Loads scale with fan-in; learning gains rescale with signal amplitude.**

7. **The precision budget is uneven but global**: gain stages (neurons) are the most mismatch-critical components per device, yet hardening them alone does not rescue the system (Section 4.6); rescue levers (area $\times$ biases) compose multiplicatively.

8. **Freeze at the peak.** Analog PC trajectories that peak and collapse (high gain, noise, mismatch) are uniformly stabilized by annealing the learning rate to zero at the peak epoch—a controller schedule, not hardware. The peak time must be measured per configuration; annealing early truncates learning instead.

9. **Cascade-gain budget.** Per-stage neuron gain must scale as $G_*^{1/\text{depth}}$ to keep total gain through the layer cascade inside a bounded stability band. Measured over four depths (rectifier net, fixed width, mirror-ratio gain in μm of output device): a one-hidden-layer net trains only at high gain (≥ 800) with no upper limit observed; the over-gain ceiling—above which the cascade over-drives into perfect anti-classification—then falls monotonically with depth (> 1600, $\sim$1600, $\sim$800, $\sim$400–800 for one through four hidden layers). Shallow nets want the gain cranked; deep nets want it turned down. (An earlier reading of these data as a fan-in law was wrong—a controlled fan-in sweep leaves the boundary fixed; depth moves it.)

10. **Mismatch yield falls with depth.** The same cascade that compounds gain compounds device offsets: a one-hidden-layer network tolerates $\sigma_{V_T} \approx 2\,\text{mV}$, while adding a second hidden layer drops the yield to $\sim$1/3 at 1 mV (deaths are mismatch-fatal—weight-init restarts and lower gain do not recover them). Depth buys representational power but costs robustness, so under mismatch the architecture preference *inverts*: shallow-and-wide beats deep-and-narrow. This sharpens the precision-budget law into a depth-dependent one.

6 Limitations

All results are simulations with idealized LEVEL-1 device models; no fabricated silicon, no noise or PVT analysis, and—as Section 4.6 quantifies—unhardened sensitivity to static mismatch. Datasets are small (2D benchmarks; 8×8 digits). Wall-clock cost is a simulation artifact: the physical system trains in milliseconds of circuit time. A faithful-but-fast surrogate under-predicted real training outcomes even at 10–30% per-step accuracy, which we read as both a caution for abstraction-based design of analog learners and evidence that the circuit exploits physics the abstraction discards.

7 Conclusion

A small library of transistor cells, wired into predictive-coding dynamics with a one-resistor soft supervision nudge and a sign-regenerating backward path, learns nonlinear tasks to high accuracy, scales to depth and to thousands of devices, runs label-free in PC's native unsupervised mode, and yields a measured set of design laws. The constraint set under which all of this works—4-bit neurons, fan-in ≤ 10, no weight transport, signs over magnitudes—is strikingly brain-like, and deliberately so: these are the operating points where analog physics is reliable. The measured mismatch wall defines the next problem precisely: the same dynamics, rebuilt from offset-canceling cells.

References

[1] B. Scellier and Y. Bengio. Equilibrium propagation: bridging the gap between energy-based models and backpropagation. *Frontiers in Computational Neuroscience*, 2017.

[2] J. Kendall, R. Pantone, K. Manickavasagam, Y. Bengio, B. Scellier. Training end-to-end analog neural networks with equilibrium propagation. arXiv:2006.01981, 2020.

[3] S. Oh et al. Memristor crossbar circuits implementing equilibrium propagation for on-device learning. *Micromachines*, 2023.

[4] (Authors at LIRMM). Designing compatible analog circuits for equilibrium propagation using the adjoint method and reciprocity principles. HAL lirmm-04959178, 2025.

[5] B. Scellier et al. Energy-based learning algorithms for analog computing: a comparative study. arXiv:2312.15103, 2023.

[6] S. Dillavou, M. Stern, A. J. Liu, D. J. Durian. Demonstration of decentralized physics-driven learning. *Phys. Rev. Applied* 18, 014040, 2022.

[7] L. G. Wright et al. Deep physical neural networks trained with backpropagation. *Nature* 601, 549, 2022.

[8] A. Momeni et al. Backpropagation-free training of deep physical neural networks. *Science*, 2023.

[9] (Authors). Fully analog end-to-end online training with real-time adaptability on an integrated photonic platform. arXiv:2506.18041, 2025.

[10] (Authors). A CMOS+X spiking neuron with on-chip machine learning. arXiv:2512.03966, 2025.

[11] G. Cauwenberghs. An analog VLSI recurrent neural network learning a continuous-time trajectory. *IEEE Trans. Neural Networks*, 1996.

[12] P. Yao et al. Fully hardware-implemented memristor convolutional neural network. *Nature* 577, 641, 2020.

[13] C. Pehle et al. The BrainScaleS-2 accelerated neuromorphic system with hybrid plasticity. *Frontiers in Neuroscience*, 2022.

[14] R. P. N. Rao and D. H. Ballard. Predictive coding in the visual cortex. *Nature Neuroscience* 2, 79, 1999.

[15] J. C. R. Whittington and R. Bogacz. An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity. *Neural Computation* 29, 1229, 2017.